

# Dossier d'Architecture Technique (DAT)

---

## 1. Table des matières

---

- 1. Table des matières
- 2. Présentation du document
  - 2.1. 1.1 Objet
  - 2.2. 1.2 Périmètre
- 3. Contexte et objectifs
  - 3.1. 2.1 Contexte
  - 3.2. 2.2 Besoins fonctionnels
  - 3.3. 2.3 Objectifs techniques
- 4. Vue d'ensemble de l'architecture
- 5. Architecture de la plateforme Data
  - 5.1. 4.1 Sources de données
    - 5.1.1. GuitarSet
    - 5.1.2. IDMT-SMT-Guitar
  - 5.2. 4.2 Architecture de stockage
    - 5.2.1. MinIO
    - 5.2.2. MongoDB
    - 5.2.3. PostgreSQL
    - 5.2.4. PostgreSQL MLflow
  - 5.3. 4.3 Calcul distribué
- 6. Architecture des pipelines de traitement
  - 6.1. 5.1 Téléchargement des jeux de données
  - 6.2. 5.2 Ingestion
  - 6.3. 5.3 Prétraitement
- 7. Exploration des données et expérimentation
  - 7.1. 6.1 Analyse exploratoire des données
  - 7.2. 6.2 Construction des jeux d'entraînement
  - 7.3. 6.3 Expérimentation des modèles
  - 7.4. 6.4 Suivi des expérimentations avec MLflow
  - 7.5. 6.5 Sélection du modèle de production
- 8. Architecture applicative
  - 8.1. 7.1 API REST
  - 8.2. 7.2 Architecture logicielle
  - 8.3. 7.3 Cycle de vie du modèle
  - 8.4. 7.4 Chaîne de traitement
  - 8.5. 7.5 Interface utilisateur
- 9. Déploiement et industrialisation
  - 9.1. 8.1 Conteneurisation de l'application
  - 9.2. 8.2 Gestion des dépendances
  - 9.3. 8.3 Déploiement local
- 10. Intégration et déploiement continu

- 10.1. 9.1 Intégration continue
  - 10.2. 9.2 Construction de l'image Docker
  - 10.3. 9.3 Déploiement continu
  - 10.4. 9.4 Plateformes de diffusion
- 11. Sécurité et gestion de la configuration
- 12. Exploitation et maintenance
- 13. Évolutions prévues
- 14. Conclusion
- 15. Annexe A — Principales décisions d'architecture
  - 15.1. Séparation entre plateforme Data et couche applicative
  - 15.2. Data Lake avec MinIO
  - 15.3. Double stockage MongoDB / PostgreSQL
    - 15.3.1. PostgreSQL
    - 15.3.2. MongoDB
  - 15.4. MLflow pour le suivi des expérimentations
  - 15.5. FastAPI
  - 15.6. Streamlit
  - 15.7. Docker
  - 15.8. uv
  - 15.9. GitHub Actions
  - 15.10. Apache Spark
- 16. Annexe B — Cartographie des diagrammes d'architecture
  - 16.1. Organisation générale des flux

## 2. Présentation du document

---

### 2.1. 1.1 Objet

Ce document constitue le **Dossier d'Architecture Technique (DAT)** du projet de transcription automatique de guitare développé dans le cadre du titre professionnel **Concepteur Développeur en Science des Données (RNCP 35288)**.

Il décrit l'architecture retenue pour concevoir, développer, industrialiser et déployer une plateforme complète de Data Science couvrant l'ensemble du cycle de vie d'un modèle d'apprentissage automatique, depuis la collecte des données jusqu'à sa mise en production.

Ce document poursuit plusieurs objectifs :

- décrire les choix d'architecture réalisés,
- présenter les différents composants techniques de la plateforme,
- expliquer les interactions entre les services,
- documenter les procédures de déploiement et d'exploitation,
- faciliter la maintenance et les évolutions futures du projet.

Il constitue le document de référence destiné aux développeurs et aux administrateurs.

### 2.2. 1.2 Périmètre

Le projet couvre l'ensemble de la chaîne de valeur d'un projet de Data Science.

La plateforme permet :

- la collecte automatisée de jeux de données audio,
- leur ingestion dans une infrastructure de stockage dédiée,
- la constitution d'un Data Lake,
- le suivi des métadonnées et du lineage des données,
- la construction automatisée d'échantillons destinés à l'apprentissage,
- l'analyse exploratoire des données,
- l'expérimentation de plusieurs modèles d'apprentissage profond,
- le suivi des expérimentations avec MLflow,
- l'industrialisation du modèle retenu,
- l'exposition du modèle sous forme d'une API REST,
- la mise à disposition d'une interface Web destinée aux utilisateurs,
- le déploiement automatisé de la couche applicative.

L'ensemble de ces composants est déployable localement grâce à Docker Compose et la couche applicative est automatiquement publiée via une chaîne d'intégration et de déploiement continus.

## 3. Contexte et objectifs

---

### 3.1. 2.1 Contexte

Le projet s'inscrit dans un contexte métier simulé inspiré d'un besoin industriel.

L'entreprise **GuitarFlow** souhaite disposer d'un service capable de convertir automatiquement un enregistrement audio de guitare en une représentation symbolique exploitable par des logiciels de musique assistée par ordinateur (MAO).

Aujourd'hui, cette opération est réalisée manuellement par des musiciens ou des ingénieurs du son. Cette étape est longue, coûteuse et nécessite une expertise importante.

L'objectif est donc de proposer une solution capable d'automatiser cette tâche tout en garantissant une qualité de transcription compatible avec une utilisation pédagogique ou musicale.

### 3.2. 2.2 Besoins fonctionnels

Le cahier des charges impose que la solution permette :

- l'import d'un fichier audio au format WAV,
- la génération d'un fichier MIDI,
- la visualisation des notes détectées sous forme de piano-roll,
- la génération d'une partition musicale lorsque cela est possible,
- l'accès au service au travers d'une API REST,
- l'utilisation du service via une interface Web simple.

### 3.3. 2.3 Objectifs techniques

Afin de répondre à ces besoins, le projet poursuit deux objectifs complémentaires.

Le premier consiste à développer une **plateforme Data** permettant de produire des jeux de données de qualité et d'expérimenter différentes architectures d'apprentissage profond dans un environnement entièrement reproductible.

Cette plateforme assure notamment :

- la collecte des données,
- leur stockage dans un Data Lake,
- le suivi des métadonnées,
- la construction des jeux d'entraînement,
- le suivi des expérimentations avec MLflow.

Le second consiste à industrialiser le modèle retenu afin de proposer un service de prédiction utilisable par des utilisateurs non spécialistes.

Cette industrialisation repose sur :

- une API REST développée avec FastAPI,
- une interface utilisateur développée avec Streamlit,
- une conteneurisation Docker,
- une chaîne d'intégration et de déploiement continu avec GitHub Actions,
- un déploiement automatisé sur Hugging Face Spaces.

L'architecture distingue volontairement la plateforme Data de la couche applicative afin de séparer les activités d'expérimentation scientifique des contraintes liées à la mise en production.

Cette séparation favorise la maintenabilité, la reproductibilité des expérimentations et l'évolution indépendante des différents composants.

## 4. Vue d'ensemble de l'architecture

---

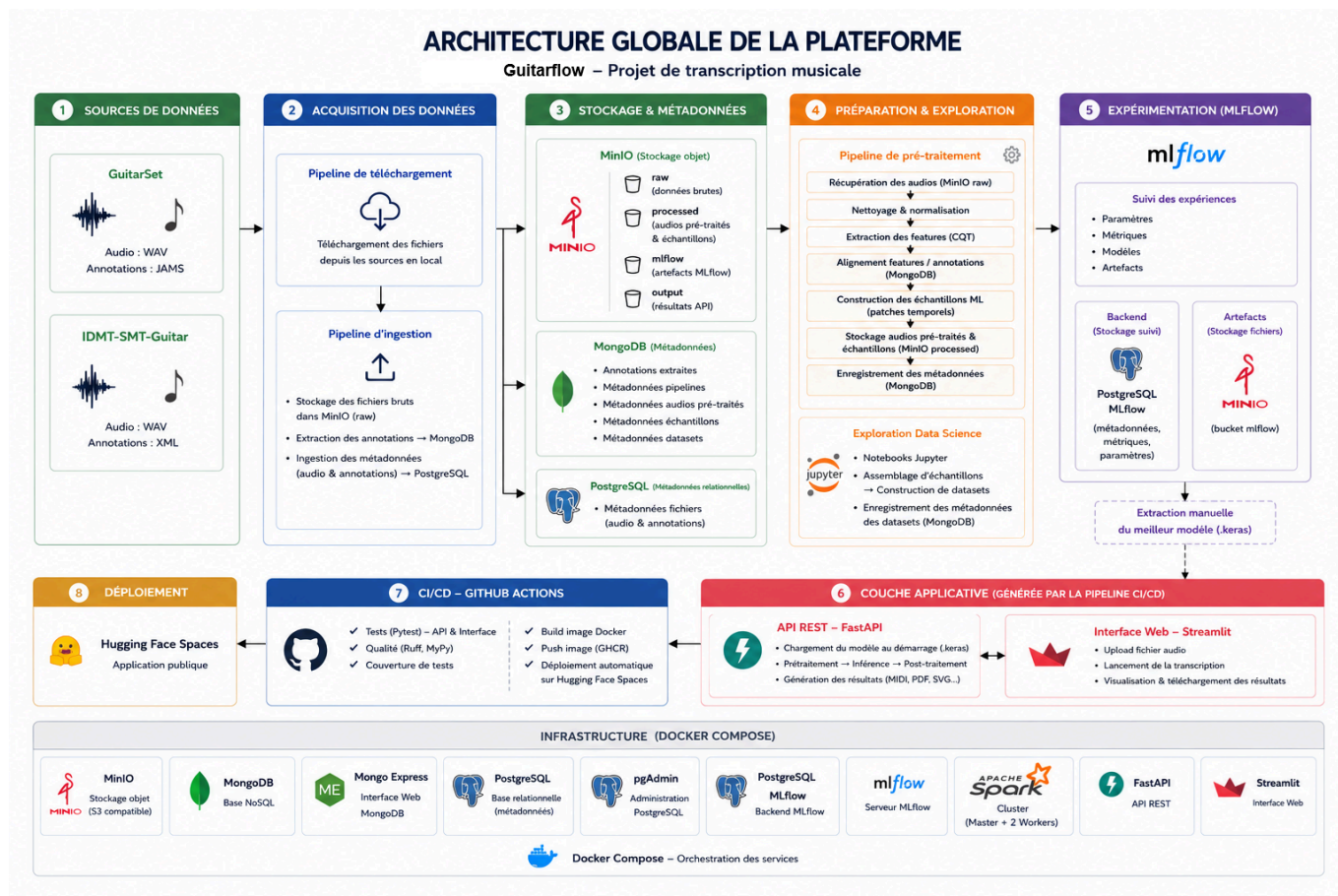
L'architecture globale de la solution est organisée autour de deux sous-systèmes complémentaires :

- une **plateforme Data**, responsable de la collecte, du stockage, du prétraitement et de l'expérimentation des modèles,
- une **couche applicative**, chargée de mettre le modèle sélectionné à disposition des utilisateurs au travers d'une API REST et d'une interface Web.

La plateforme Data constitue le socle technique du projet. Elle permet de produire des données d'entraînement de qualité, de garantir leur traçabilité et de comparer plusieurs architectures de réseaux de neurones grâce au suivi des expérimentations réalisé par MLflow.

La couche applicative exploite uniquement le modèle retenu. Celui-ci est exporté manuellement depuis MLflow, intégré au dépôt Git puis chargé au démarrage de l'API. Aucune phase d'entraînement n'est réalisée en production, l'application se limite aux opérations de prétraitement, d'inférence et de génération des artefacts destinés à l'utilisateur.

La figure ci-dessous présente l'architecture générale de la plateforme.



## 5. Architecture de la plateforme Data

La plateforme Data constitue le cœur du projet. Elle regroupe l'ensemble des composants nécessaires à la collecte, au stockage, à la préparation des données ainsi qu'à l'expérimentation des modèles d'apprentissage automatique.

Son architecture a été conçue selon une logique modulaire afin de séparer les différentes responsabilités (ingestion, stockage, prétraitement, expérimentation), de faciliter la maintenance et de garantir la reproductibilité des traitements.

### 5.1. 4.1 Sources de données

Le projet s'appuie sur deux jeux de données publics spécialisés dans la transcription automatique de guitare.

#### 5.1.1. GuitarSet

Le jeu de données **GuitarSet** fournit :

- des enregistrements audio au format **WAV**,
- des annotations musicales au format **JAMS**.

Ces données couvrent plusieurs styles musicaux et techniques de jeu, ce qui permet d'obtenir un corpus représentatif des situations rencontrées lors de l'entraînement.

#### 5.1.2. IDMT-SMT-Guitar

Le jeu de données **IDMT-SMT-Guitar** fournit :

- des fichiers audio au format **WAV**,
- des annotations musicales au format **XML**.

Il complète GuitarSet en apportant des exemples supplémentaires ainsi que plusieurs sous-ensembles présentant des caractéristiques différentes.

L'utilisation simultanée de ces deux jeux de données améliore la diversité des données d'entraînement et limite les risques de sur-apprentissage.

## 5.2. 4.2 Architecture de stockage

Afin de séparer les différents types d'informations manipulées par la plateforme, plusieurs technologies de stockage sont utilisées.

### 5.2.1. MinIO

MinIO joue le rôle de **Data Lake**.

Il centralise les objets volumineux produits tout au long du projet.

Quatre buckets sont utilisés :

| Bucket           | Contenu                                                           |
|------------------|-------------------------------------------------------------------|
| <b>raw</b>       | données brutes téléchargées (audio et annotations)                |
| <b>processed</b> | données prétraitées, features et échantillons d'apprentissage     |
| <b>mlflow</b>    | artefacts produits par MLflow (modèles, figures, fichiers divers) |
| <b>output</b>    | réservé aux futurs artefacts produits par l'API                   |

Cette organisation permet de distinguer clairement les différentes étapes du cycle de vie des données.

### 5.2.2. MongoDB

MongoDB assure la gestion des métadonnées de la plateforme.

Les principales informations enregistrées sont :

- les annotations extraites des jeux de données,
- les métadonnées des pipelines d'ingestion,
- les métadonnées des pipelines de prétraitement,
- les informations décrivant les échantillons générés,
- les jeux de données d'entraînement construits,
- les relations entre les différents objets stockés.

MongoDB constitue ainsi le référentiel de **lineage** de la plateforme en permettant de retracer l'origine des données utilisées lors des expérimentations.

### 5.2.3. PostgreSQL

Une première base PostgreSQL stocke les métadonnées relationnelles des fichiers audio et des annotations.

Cette séparation entre PostgreSQL et MongoDB permet de tirer parti des avantages respectifs des deux modèles de données :

- base relationnelle pour les informations fortement structurées,
- base documentaire pour les métadonnées évolutives des pipelines.

#### 5.2.4. PostgreSQL MLflow

Une seconde instance PostgreSQL est dédiée exclusivement au suivi des expérimentations MLflow.

Elle stocke notamment :

- les expériences,
- les exécutions (runs),
- les paramètres,
- les métriques,
- les références vers les artefacts.

Les fichiers associés aux expérimentations sont quant à eux enregistrés dans le bucket **mlflow** de MinIO.

### 5.3. 4.3 Calcul distribué

L'infrastructure intègre un cluster Apache Spark composé :

- d'un noeud maître,
- de deux noeuds de calcul.

Cette infrastructure est pleinement déployée par Docker Compose.

À ce stade du projet, les pipelines de préparation des données sont encore implémentés en Python. Le cluster Spark n'est donc pas utilisé en production.

Son intégration répond néanmoins à un objectif d'évolutivité. Une version ultérieure de la plateforme prévoit de migrer les traitements de prétraitement audio vers Spark afin d'accélérer la génération des caractéristiques sur des volumes de données plus importants.

Cette anticipation permet de démontrer que l'architecture a été conçue pour supporter une montée en charge sans remise en cause de son organisation générale.

## 6. Architecture des pipelines de traitement

---

Les traitements de préparation des données sont organisés sous forme de pipelines indépendants.

Chaque pipeline possède une responsabilité unique et produit les données nécessaires au pipeline suivant.

Cette organisation favorise :

- la réutilisation des traitements,
- la reproductibilité,

- la maintenance,
- la traçabilité des opérations.

Le traitement complet des données est constitué de trois grandes étapes successives.

## 6.1. 5.1 Téléchargement des jeux de données

Le premier pipeline automatise le téléchargement des jeux de données publics.

Il récupère :

- GuitarSet,
- IDMT-SMT-Guitar.

Les fichiers sont téléchargés localement avant toute opération d'ingestion.

Cette étape permet de conserver une copie des données sources indépendamment des traitements réalisés ensuite.

## 6.2. 5.2 Ingestion

Le pipeline d'ingestion importe les données brutes dans l'infrastructure de stockage.

Pour chaque jeu de données, il réalise les opérations suivantes :

- import des fichiers audio dans le bucket **raw** de MinIO,
- extraction des annotations musicales,
- stockage des annotations dans MongoDB,
- enregistrement des métadonnées des fichiers dans PostgreSQL.

À l'issue de cette étape, les données sources sont entièrement intégrées à la plateforme.

## 6.3. 5.3 Prétraitement

Le pipeline de prétraitement prépare les données destinées à l'apprentissage automatique.

Les principales opérations réalisées sont :

- chargement des fichiers audio depuis MinIO,
- nettoyage et normalisation des signaux audio,
- extraction des caractéristiques fréquentielles (features),
- récupération des annotations depuis MongoDB,
- alignement temporel entre les caractéristiques et les annotations,
- génération des échantillons d'apprentissage.

Les fichiers produits sont ensuite enregistrés dans le bucket **processed**.

Les métadonnées des traitements ainsi que les informations décrivant les échantillons générés sont enregistrées dans MongoDB afin de garantir leur traçabilité.

Cette architecture permet de reconstruire intégralement un jeu de données d'entraînement à partir des informations enregistrées par la plateforme.



## 7. Exploration des données et expérimentation

---

Une fois les données intégrées et prétraitées, la plateforme permet de réaliser les analyses exploratoires ainsi que les expérimentations nécessaires à la conception du modèle d'apprentissage automatique.

Cette étape constitue le socle scientifique du projet. Elle permet de comprendre les caractéristiques des données, de construire les jeux d'entraînement et de comparer plusieurs architectures de réseaux de neurones avant leur mise en production.

### 7.1. 6.1 Analyse exploratoire des données

Trois analyses exploratoires (EDA) ont été réalisées au cours du projet.

La première porte sur le jeu de données **GuitarSet**, la deuxième sur **IDMT-SMT-Guitar**, tandis que la troisième analyse le jeu d'entraînement construit à partir des échantillons générés par la plateforme.

Ces analyses poursuivent plusieurs objectifs :

- évaluer la qualité des données collectées,
- identifier les éventuelles anomalies ou incohérences,
- caractériser la répartition des observations,
- vérifier la diversité des notes et des techniques de jeu,
- analyser les distributions statistiques des variables utilisées pour l'apprentissage.

Les résultats sont présentés sous forme de graphiques et de statistiques descriptives produits à l'aide des bibliothèques Pandas, NumPy et Matplotlib.

Cette phase permet de valider la qualité des données avant leur utilisation pour l'entraînement des modèles.

### 7.2. 6.2 Construction des jeux d'entraînement

Les expérimentations ne sont pas réalisées directement sur les fichiers audio.

Les pipelines de prétraitement produisent des échantillons indépendants stockés dans le bucket **processed** de MinIO.

À partir de ces échantillons, plusieurs jeux d'entraînement peuvent être construits afin de répondre à différents besoins expérimentaux.

Chaque dataset est défini par une sélection d'échantillons dont les métadonnées sont enregistrées dans MongoDB.

Cette approche présente plusieurs avantages :

- éviter de reconstruire les échantillons à chaque expérimentation,
- garantir la reproductibilité des jeux de données,
- faciliter la comparaison entre plusieurs modèles,
- assurer la traçabilité complète des données utilisées lors de l'entraînement.

### 7.3. 6.3 Expérimentation des modèles

L'entraînement des modèles est réalisé dans des notebooks Jupyter.

Cette organisation favorise l'exploration interactive des données ainsi que l'évaluation de plusieurs architectures d'apprentissage profond.

Au cours du projet, plusieurs approches ont été étudiées, notamment :

- One-vs-Rest associé à HistGradientBoosting,
- perceptron multicouche (MLP),
- réseau de neurones convolutif (CNN) associé à un MLP,
- réseau récurrent convolutif (RCNN).

Chaque expérimentation est conduite indépendamment afin d'évaluer les performances des différentes architectures dans des conditions identiques.

Le choix du modèle final repose sur l'analyse comparative des métriques obtenues.

## 7.4. 6.4 Suivi des expérimentations avec MLflow

L'ensemble des expérimentations est suivi à l'aide de MLflow.

Pour chaque entraînement, la plateforme enregistre automatiquement :

- les paramètres d'entraînement,
- les hyperparamètres,
- les métriques d'évaluation,
- les artefacts produits (figures, modèles, fichiers divers).

Les métadonnées des expériences sont stockées dans une base PostgreSQL dédiée tandis que les artefacts sont enregistrés dans le bucket **mlflow** de MinIO.

Cette organisation garantit la reproductibilité des expérimentations et facilite la comparaison entre plusieurs versions d'un même modèle.

## 7.5. 6.5 Sélection du modèle de production

À l'issue des expérimentations, le modèle présentant les meilleures performances est retenu pour être mis en production.

Le modèle exporté est un modèle TensorFlow au format **.keras**.

Son intégration dans la couche applicative est réalisée manuellement afin de conserver une parfaite maîtrise des versions déployées.

Cette étape constitue la frontière entre la plateforme Data et la couche applicative.

Aucun entraînement n'est réalisé dans l'application de production.

Le modèle est uniquement chargé au démarrage de l'API puis partagé entre l'ensemble des requêtes.

# 8. Architecture applicative

---

La couche applicative constitue la partie visible du projet.

Elle met le modèle d'apprentissage automatique à disposition des utilisateurs au travers d'une API REST et d'une interface Web.

Son architecture repose sur une séparation claire des responsabilités afin de faciliter les évolutions, les tests et la maintenance.

Elle est composée de deux applications complémentaires :

- une API REST développée avec **FastAPI**,
- une interface utilisateur développée avec **Streamlit**.

Les deux applications sont exécutées simultanément au sein d'un même conteneur Docker.

## 8.1. 7.1 API REST

L'API constitue le point d'entrée principal de la plateforme.

Elle reçoit les fichiers audio transmis par les utilisateurs, orchestre l'ensemble des traitements nécessaires puis retourne les artefacts générés.

Les principales fonctionnalités exposées sont :

- vérification de l'état de santé de l'application,
- consultation des informations du modèle chargé,
- lancement d'une transcription,
- téléchargement des artefacts générés.

Les réponses sont normalisées grâce à un modèle générique garantissant une structure homogène sur l'ensemble des routes.

La documentation OpenAPI est générée automatiquement par FastAPI.

## 8.2. 7.2 Architecture logicielle

L'API est organisée selon une architecture en couches distinguant les responsabilités techniques et fonctionnelles.

Les principaux composants sont :

- les routes HTTP, responsables de la communication avec les clients,
- les services métier, qui implémentent les traitements,
- les modèles de données utilisés pour les échanges,
- les dépendances injectées par FastAPI,
- les composants techniques assurant notamment le chargement du modèle et la configuration de l'application.

Cette organisation limite le couplage entre les composants et facilite les tests unitaires.

## 8.3. 7.3 Cycle de vie du modèle

Le modèle TensorFlow est chargé une seule fois au démarrage de l'application grâce au mécanisme **lifespan** de FastAPI.

Cette stratégie évite le rechargement du modèle à chaque requête et réduit significativement le temps de réponse de l'application.

Le gestionnaire de modèle centralise notamment :

- le modèle TensorFlow,
- les métadonnées du modèle,
- les paramètres d'inférence.

Ces ressources sont ensuite partagées par l'ensemble des services de prédiction.

## 8.4. 7.4 Chaîne de traitement

Lorsqu'un utilisateur soumet un fichier audio, l'application exécute successivement les étapes suivantes :

1. validation du fichier reçu,
2. prétraitement du signal audio,
3. extraction des caractéristiques nécessaires au modèle,
4. inférence par le réseau de neurones,
5. post-traitement des prédictions,
6. génération des différents artefacts.

Les résultats produits comprennent notamment :

- un fichier MIDI,
- un piano-roll aux formats SVG et PNG,
- une partition musicale aux formats SVG et PDF lorsque sa génération est possible.

Cette chaîne de traitement reprend les mêmes principes que ceux utilisés lors de la préparation des données d'entraînement, garantissant ainsi la cohérence entre les phases d'apprentissage et de production.

## 8.5. 7.5 Interface utilisateur

L'application Streamlit fournit une interface graphique destinée aux utilisateurs ne souhaitant pas interagir directement avec l'API REST.

Elle permet :

- le dépôt d'un fichier audio au format WAV,
- le lancement de la transcription,
- la consultation des résultats,
- le téléchargement des artefacts générés.

L'interface communique exclusivement avec l'API REST.

Cette séparation permet de faire évoluer indépendamment l'interface utilisateur et les services métier tout en conservant une architecture cohérente.

## 9. Déploiement et industrialisation

---

L'industrialisation de la solution vise à garantir qu'un même code source puisse être exécuté de manière identique en développement, en intégration continue et en production.

Cette reproductibilité repose sur quatre piliers :

- la conteneurisation de l'application avec Docker,
- la gestion déterministe des dépendances Python avec **uv**,
- l'automatisation des contrôles qualité avec GitHub Actions,
- le déploiement continu sur Hugging Face Spaces.

### 9.1. 8.1 Conteneurisation de l'application

La couche applicative est distribuée sous la forme d'une image Docker unique.

Le conteneur regroupe l'ensemble des composants nécessaires à l'exécution de l'application :

- Python 3.13,
- FastAPI,
- Streamlit,
- TensorFlow,
- music21,
- Verovio,
- CairoSVG,
- les dépendances système nécessaires au traitement audio et à la génération des partitions.

Le démarrage du conteneur est assuré par le script **start.sh**, qui lance simultanément :

- l'API FastAPI sur le port **8000**,
- l'interface Streamlit sur le port **7860**.

Cette approche garantit que tous les environnements exécutent exactement la même version de l'application.

### 9.2. 8.2 Gestion des dépendances

Le projet utilise **uv** comme gestionnaire de dépendances Python.

L'ensemble des bibliothèques est décrit dans le fichier **pyproject.toml**, tandis que le fichier **uv.lock** verrouille précisément les versions installées.

Cette stratégie présente plusieurs avantages :

- installations déterministes,
- reproductibilité des environnements,
- réduction des différences entre développement et production,
- simplification de l'intégration continue.

### 9.3. 8.3 Déploiement local

L'ensemble de la plateforme peut être démarré localement grâce au fichier `docker-compose.yml`.

Cette infrastructure déploie notamment :

- MinIO,
- MongoDB,
- Mongo Express,
- PostgreSQL,
- pgAdmin,
- PostgreSQL dédié à MLflow,
- MLflow,
- un cluster Apache Spark,
- l'API FastAPI,
- l'interface Streamlit.

Chaque service est isolé dans son propre réseau Docker et plusieurs composants disposent d'une vérification d'état (health check) garantissant un démarrage ordonné de la plateforme.

Une fois l'infrastructure démarrée, les principaux services sont accessibles via un navigateur :

| Service               | Adresse                                                             |
|-----------------------|---------------------------------------------------------------------|
| API REST              | <a href="http://localhost:8000">http://localhost:8000</a>           |
| Documentation OpenAPI | <a href="http://localhost:8000/docs">http://localhost:8000/docs</a> |
| Interface Streamlit   | <a href="http://localhost:7860">http://localhost:7860</a>           |
| MLflow                | <a href="http://localhost:5000">http://localhost:5000</a>           |
| MinIO Console         | <a href="http://localhost:9001">http://localhost:9001</a>           |
| pgAdmin               | <a href="http://localhost:8080">http://localhost:8080</a>           |
| Mongo Express         | <a href="http://localhost:8081">http://localhost:8081</a>           |

Le fichier `.env.example` fournit l'ensemble des variables nécessaires à la configuration de l'environnement.

## 10. Intégration et déploiement continu

---

La couche applicative est automatiquement validée puis déployée grâce à une chaîne CI/CD développée avec GitHub Actions.

Cette automatisation permet de sécuriser les mises en production tout en limitant les interventions manuelles.

### 10.1. 9.1 Intégration continue

Chaque `push` ou `pull request` déclenche automatiquement un workflow GitHub Actions.

La phase d'intégration continue réalise successivement :

- l'installation d'un environnement Python reproductible avec uv,

- l'exécution des tests unitaires et d'intégration avec Pytest,
- la mesure de la couverture de tests,
- l'analyse statique avec Ruff,
- la vérification du typage avec MyPy.

Les rapports de couverture sont publiés sous forme d'artefacts GitHub Actions.

Cette étape garantit qu'aucune régression fonctionnelle ou de qualité n'est introduite avant le déploiement.

## 10.2. 9.2 Construction de l'image Docker

Après validation des contrôles qualité, GitHub Actions construit automatiquement l'image Docker de l'application.

Les métadonnées de l'image (nom, version, tags Git, SHA du commit) sont générées automatiquement.

L'image est ensuite publiée dans **GitHub Container Registry (GHCR)**.

Cette image constitue l'unique artefact de déploiement de la couche applicative.

## 10.3. 9.3 Déploiement continu

Le déploiement de l'application est entièrement automatisé.

À chaque mise à jour de la branche principale :

1. une nouvelle image Docker est publiée dans GHCR,
2. GitHub Actions met à jour le Dockerfile du Space Hugging Face,
3. Hugging Face détecte automatiquement la modification,
4. une nouvelle version de l'application est reconstruite puis déployée.

Cette stratégie garantit que la version disponible en ligne correspond toujours au dernier état validé du dépôt GitHub.

## 10.4. 9.4 Plateformes de diffusion

Le code source est hébergé sur GitHub.

L'image Docker est publiée sur GitHub Container Registry.

L'application est déployée automatiquement sur Hugging Face Spaces, qui exécute directement l'image Docker publiée.

Cette architecture évite toute reconstruction spécifique sur la plateforme d'hébergement et garantit une parfaite cohérence entre les différents environnements d'exécution.

# 11. Sécurité et gestion de la configuration

---

La plateforme applique plusieurs principes visant à sécuriser son exécution.

Les paramètres de configuration sont externalisés dans des variables d'environnement.

Les identifiants de production (jetons GitHub, Hugging Face, mots de passe des services) ne sont jamais présents dans le code source.

Le dépôt fournit uniquement un fichier `.env.example` permettant de reproduire l'environnement sans divulguer d'informations sensibles.

Les secrets utilisés par la chaîne CI/CD sont stockés dans GitHub Secrets.

Enfin, l'application Docker est exécutée avec un utilisateur dédié afin de limiter les privilèges accordés au conteneur.

## 12. Exploitation et maintenance

---

L'architecture retenue facilite les opérations d'exploitation.

La séparation entre plateforme Data et couche applicative permet de faire évoluer indépendamment les traitements de préparation des données, les expérimentations et l'application de production.

Le découpage de l'API en couches (routes, services, dépendances, modèles et composants techniques) améliore la lisibilité du code et simplifie les opérations de maintenance.

La présence de tests automatisés et d'une chaîne CI/CD permet de détecter rapidement les régressions lors des évolutions futures.

## 13. Évolutions prévues

---

L'architecture a été conçue afin de faciliter les évolutions du projet.

Plusieurs améliorations sont envisagées :

- migration des pipelines de prétraitement vers Apache Spark afin de paralléliser les traitements,
- automatisation de la récupération du meilleur modèle depuis MLflow,
- exploitation du bucket `output` de MinIO pour conserver les artefacts produits par l'API,
- enrichissement des fonctionnalités de l'interface utilisateur,
- ajout de nouvelles architectures d'apprentissage profond,
- mise en place d'une supervision centralisée de l'application.

Ces évolutions pourront être intégrées sans remise en cause de l'architecture générale grâce au découpage modulaire de la plateforme.

## 14. Conclusion

---

Le projet met en œuvre une architecture complète couvrant l'ensemble du cycle de vie d'un projet de Data Science.

La plateforme assure la collecte des données, leur stockage, leur préparation, l'expérimentation des modèles, puis l'industrialisation du modèle retenu au travers d'une API REST et d'une interface Web.



Les choix d'architecture retenus privilégient la modularité, la reproductibilité et l'automatisation. La conteneurisation, la gestion déterministe des dépendances, l'intégration continue et le déploiement automatisé garantissent la cohérence des environnements et facilitent la maintenance de la solution.

Cette architecture répond ainsi aux exigences d'un projet de mise en production d'un algorithme d'apprentissage automatique tout en restant suffisamment évolutive pour accompagner les développements futurs.

## 15. Annexe A — Principales décisions d'architecture

---

Cette section synthétise les principaux choix d'architecture réalisés au cours du projet ainsi que leur justification.

L'objectif est de montrer que les technologies retenues résultent d'une analyse des besoins fonctionnels et techniques, et non d'un simple choix d'outils.

### 15.1. Séparation entre plateforme Data et couche applicative

L'architecture distingue volontairement deux ensembles indépendants :

- une **plateforme Data**, dédiée à la préparation des données, aux analyses exploratoires et aux expérimentations,
- une **couche applicative**, dédiée à l'exploitation du modèle sélectionné.

Cette séparation présente plusieurs avantages :

- indépendance entre la recherche et la production,
- réduction des risques lors des mises à jour du modèle,
- meilleure maintenabilité,
- possibilité de faire évoluer les pipelines sans impacter l'application déployée.

Cette organisation s'inspire des architectures MLOps généralement utilisées en entreprise.

### 15.2. Data Lake avec MinIO

Le stockage des fichiers volumineux repose sur MinIO.

Ce choix permet :

- de disposer d'un stockage objet compatible avec Amazon S3,
- de séparer les fichiers des métadonnées,
- de conserver plusieurs versions des données,
- de faciliter une éventuelle migration vers un service cloud.

Les buckets sont organisés selon le cycle de vie des données :

- **raw**
- **processed**
- **mlflow**
- **output**

Cette organisation améliore la lisibilité de la plateforme et facilite la gouvernance des données.

## 15.3. Double stockage MongoDB / PostgreSQL

Le projet utilise volontairement deux technologies de bases de données.

### 15.3.1. PostgreSQL

PostgreSQL stocke les données fortement structurées nécessitant des relations entre objets.

Il est utilisé notamment pour :

- les métadonnées des fichiers audio,
- les informations relatives aux annotations,
- la base de données de MLflow.

### 15.3.2. MongoDB

MongoDB est utilisé pour les données documentaires dont le schéma évolue au fil des pipelines.

Il stocke notamment :

- les annotations extraites,
- les métadonnées des pipelines,
- les métadonnées des audios prétraités,
- les métadonnées des échantillons,
- les métadonnées des datasets.

Cette combinaison permet d'utiliser chaque technologie dans son domaine de pertinence.

## 15.4. MLflow pour le suivi des expérimentations

Toutes les expérimentations sont suivies avec MLflow.

Ce choix garantit :

- la reproductibilité des entraînements,
- la comparaison des modèles,
- la conservation des métriques,
- le stockage des artefacts.

Le modèle intégré dans l'application est sélectionné à partir des résultats enregistrés dans MLflow.

## 15.5. FastAPI

L'API REST est développée avec FastAPI.

Ce framework a été retenu pour :

- ses performances,
- son typage Python natif,
- la génération automatique de la documentation OpenAPI,

- son excellente intégration avec Pydantic.

Ces caractéristiques facilitent la maintenance ainsi que la création d'une API robuste.

## 15.6. Streamlit

L'interface utilisateur est développée avec Streamlit.

L'objectif n'est pas de construire une application Web complexe mais de proposer une interface simple permettant :

- le dépôt d'un fichier audio,
- l'exécution d'une transcription,
- le téléchargement des résultats.

Cette approche permet de démontrer rapidement le fonctionnement du modèle auprès d'utilisateurs non techniques.

## 15.7. Docker

Toute la couche applicative est distribuée sous la forme d'une image Docker unique.

Cette stratégie garantit :

- un environnement identique entre développement et production,
- une installation simplifiée,
- un déploiement reproductible,
- une meilleure portabilité de l'application.

Le même conteneur est utilisé localement, dans GitHub Actions et sur Hugging Face Spaces.

## 15.8. uv

Le projet utilise **uv** pour la gestion des dépendances Python.

Ce choix permet :

- une installation rapide,
- un verrouillage précis des versions,
- des environnements déterministes,
- une meilleure reproductibilité des développements.

Le fichier **uv.lock** garantit que toutes les plateformes utilisent exactement les mêmes versions de bibliothèques.

## 15.9. GitHub Actions

La chaîne CI/CD automatise :

- les tests,
- les analyses statiques,
- la construction de l'image Docker,

- la publication sur GitHub Container Registry,
- le déploiement sur Hugging Face.

Cette automatisation réduit fortement les erreurs humaines lors des mises en production.

## 15.10. Apache Spark

L'infrastructure intègre un cluster Spark composé d'un nœud maître et de deux nœuds de calcul.

À ce stade, les traitements sont encore réalisés en Python.

Spark a néanmoins été intégré dès la conception afin de préparer une future montée en charge de la plateforme.

L'architecture retenue permettra de migrer progressivement les pipelines de prétraitement vers un traitement distribué sans remettre en cause les autres composants de la plateforme.

## 16. Annexe B — Cartographie des diagrammes d'architecture

---

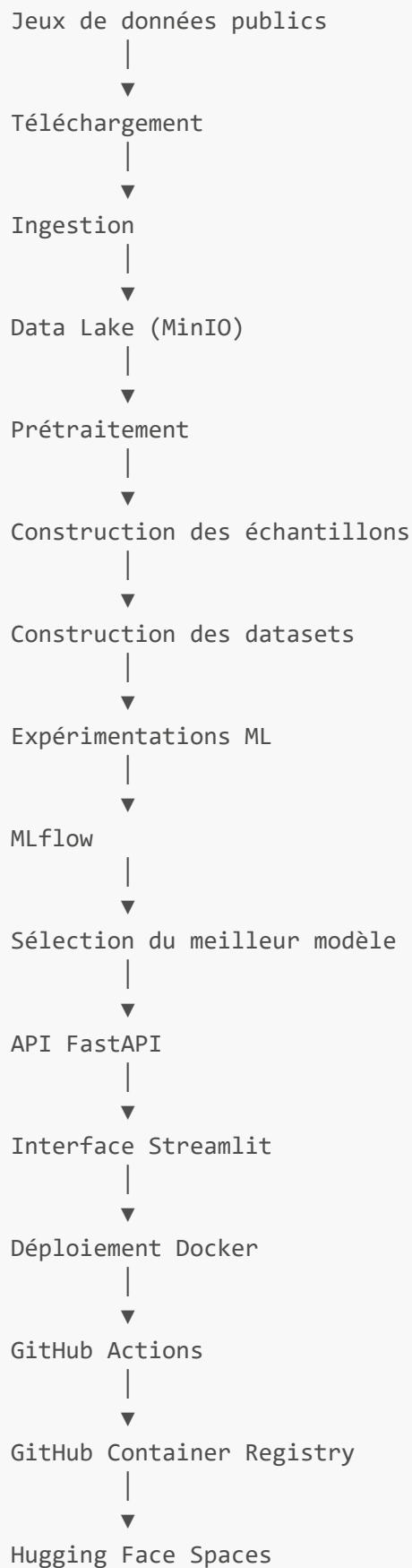
Afin de faciliter la compréhension du projet, plusieurs diagrammes complètent ce document.

Chaque diagramme illustre un niveau d'abstraction différent de l'architecture.

| Figure          | Description                                                            | Emplacement                                                                        |
|-----------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Figure 1</b> | Architecture globale de la plateforme Data et de la couche applicative | <a href="#">livrables/soutenance/figures/BC01/architecture_globale.png</a>         |
| <b>Figure 2</b> | Architecture de déploiement (GitHub, CI/CD, GHCR, Hugging Face)        | <a href="#">livrables/soutenance/figures/BC05/cicd_deployment_architecture.png</a> |
| <b>Figure 3</b> | MLD des métadonnées                                                    | <a href="#">livrables/gestion_de_projet/MLD.png</a>                                |

### 16.1. Organisation générale des flux

L'architecture suit le cycle de vie classique d'un projet de Data Science :



Cette représentation synthétise l'ensemble du cycle de vie couvert par le projet, depuis la collecte des données jusqu'au déploiement automatisé de l'application.

**Auteur :** Damien DESSAUX